

# Doubleword `/v1/files` Malformed Upload — Postmortem & Fix

Author: Derek J. Russell · Project: JARVIS Trinity AI / Ouroboros · Date: 2026-05-28

Reference: Doubleword Support email "*Malformed File Uploads*" · peter@doubleword.ai · 2026-05-28 10:23 AM

---

## Summary

Our batch upload path was serializing the JSONL request body with **no trailing newline**. The result was structurally valid JSON but structurally invalid JSONL (*ndjson*, RFC 7464). The Doubleword `/v1/files` validator correctly rejected every upload with HTTP 500 "*Internal server error*" — matching the "*invalid multi part files*" classification Peter observed on the Doubleword side.

**Fix:** append a single `\n` after `json.dumps(...)`. Shipped 2026-05-28 as Slice 38 of our governance pipeline (canonical composer + AST-pinned regression guards). Empirically verified: a single isolated upload against `/v1/files` now returns HTTP 200 with a real `file_id` in 0.71 s.

## Root cause

Two call sites in `doubleword_provider.py` built the upload payload as:

```
jsonl_line = json.dumps({
    "custom_id": operation_id,
    "method": "POST",
    "url": "/v1/chat/completions",
    "body": { ... },
})
await self._upload_file(jsonl_line, op_id=operation_id)
```

`json.dumps` emits a single JSON object terminated by `}` with no line terminator. RFC 7464 / *ndjson* requires every record to be terminated by `\n`. A single object without the terminator is not a valid JSONL file — the parser cannot distinguish "the record ended" from "more bytes coming." Doubleword's validator rejected each upload.

## Empirical evidence

Reproduced on a representative payload (Qwen3.5-35B-A3B-FP8, 1-message body):

|             | Pre-fix                            | Post-fix (Slice 38)                           |
|-------------|------------------------------------|-----------------------------------------------|
| bytes       | 357                                | 358                                           |
| last byte   | }                                  | \n                                            |
| ends \n     | False                              | True                                          |
| DW verdict  | HTTP 500                           | HTTP 200                                      |
| DW response | "Internal server error" (21 bytes) | file_id: dcb7e506-02c3-4c20-8618-1c15cc1504cc |
| round-trip  | n/a (rejected)                     | 0.71 s                                        |

Across our 2026-05-28 capability soak (session `bt-2026-05-28-162729`), **3 out of 3** uploads were rejected with the same shape — payload sizes 4 KB / 18 KB / 33 KB all failed identically, confirming the failure was payload-size-independent and model-specific to the malformation, not capacity.

## The fix

We introduced a single canonical helper as the only path for batch-entry JSONL construction:

```
@staticmethod
def _compose_jsonl_batch_entry(entry: dict) -> str:
    """RFC 7464 ndjson – exactly one trailing newline."""
    # ... required-field validation elided ...
    return json.dumps(entry) + "\n"
```

Three layers of defense ensure this cannot regress:

- 1. Single source of truth.** Both production call sites (batch + prompt-only) call the composer; raw `json.dumps` is gone from the upload path.
- 2. Belt-and-braces guard.** `_upload_file` itself checks `jsonl_content.endswith("\n")` and warn-and-fixes any caller that bypasses the composer.
- 3. AST-pinned regression test** rejects any future `json.dumps(...)` → `_upload_file(...)` chain at PR review time.

## Verification

Single-call smoke test against your `/v1/files` endpoint, 2026-05-28 11:05 AM PT:

```
[DoublewordProvider] File upload START: payload=278 bytes
  custom_id=v34-gatekick-1779991517
  model=Qwen/Qwen3.5-35B-A3B-FP8
```

```
[PreflightGate] Trailing newline fix accepted by
  Doubleword gateway. Proceeding to unconstrained
  capability soak.
```

```
file_id   = dcb7e506-02c3-4c20-8618-1c15cc1504cc
status    = 200 (no error)
elapsed   = 0.71 s
```

13 new unit tests + 173 cross-arc regression tests all green on our side. We'll continue to monitor capability runs to make sure no other upload-layer issues surface.

---

If you'd like a sample of the pre-fix (malformed) JSONL payload for your validator's test corpus, we're happy to send one separately. Otherwise this should be quiet from our side.

**Slice 38 commit:** 2594b2c38e on main · PR #63660